LABORATORY 06 – Application Layer Protocols, Base Platform, and Network Layer

AUTORS

Andrés Felipe Chavarro Plazas

David Santiago Espinosa Rojas

TEACHER

Diego Alejandro Murcia Cespedes

ESCUELA COLOMBIANA DE INGENIERÍA JULIO GARAVITO

VIGILADA MINEDUCACIÓN

COMPUTER NETWORKS

GROUP 01

Bogota, Colombia

April 06, 2025

OBJETIVE

For this laboratory, we are going to install and configure basic web server software and configure routers in Packet Tracer to establish connectivity between devices on the network.

MATERIALS & TOOLS

- Laboratory computers with internet access.
- Virtualization software.

INTRODUCTION

In this lab, we will work on configuring the network infrastructure and web servers. First, we will use Packet Tracer to simulate and configure routers, establishing connectivity between different networks using static addressing. Then, we will install and configure web servers on various operating systems, ensuring their accessibility from other devices on the network. Finally, we will integrate a DNS service to allow access to web servers by name instead of by IP address.

THEORETICAL FRAMEWORK

In a network infrastructure, the configuration of interconnection devices, such as routers and switches, is critical to ensuring efficient communication between different network segments. Routers allow the interconnection of multiple networks by routing packets, using static or dynamic route tables. To simulate these configurations, tools such as Packet Tracer allow replicating network scenarios with routers, switches, and computers, facilitating the testing and validation of configurations before implementing them in real world environments.

On the other hand, web servers are an essential component of IT infrastructure, as they allow the hosting and distribution of content accessible via the Internet or internal networks. Among the most commonly used web servers are Apache and Nginx, which provide HTTP and HTTPS services for delivering web pages and applications. Their installation and configuration on different operating systems allow for evaluating their performance and optimization in specific environments.

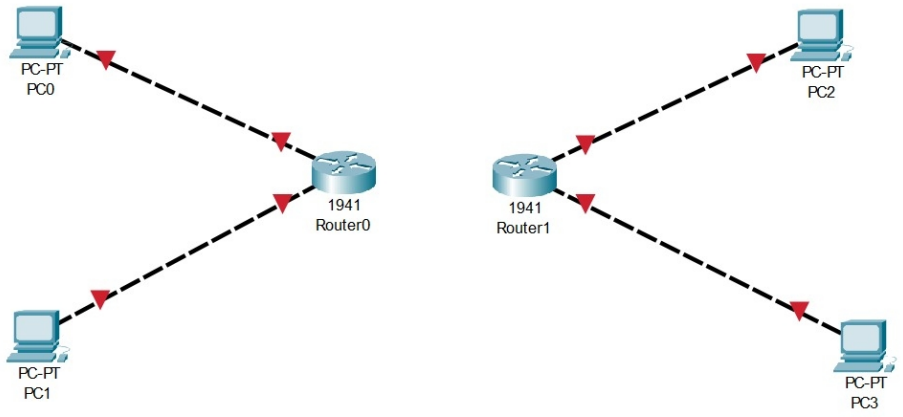
Furthermore, the DNS is key to improving the accessibility and usability of web servers, as it allows users to access resources using domain names instead of IP addresses. Setting up a DNS server within your network makes it easier to manage internal and external domain names, ensuring that clients can correctly resolve the addresses of available web services.

SIMULATIONS

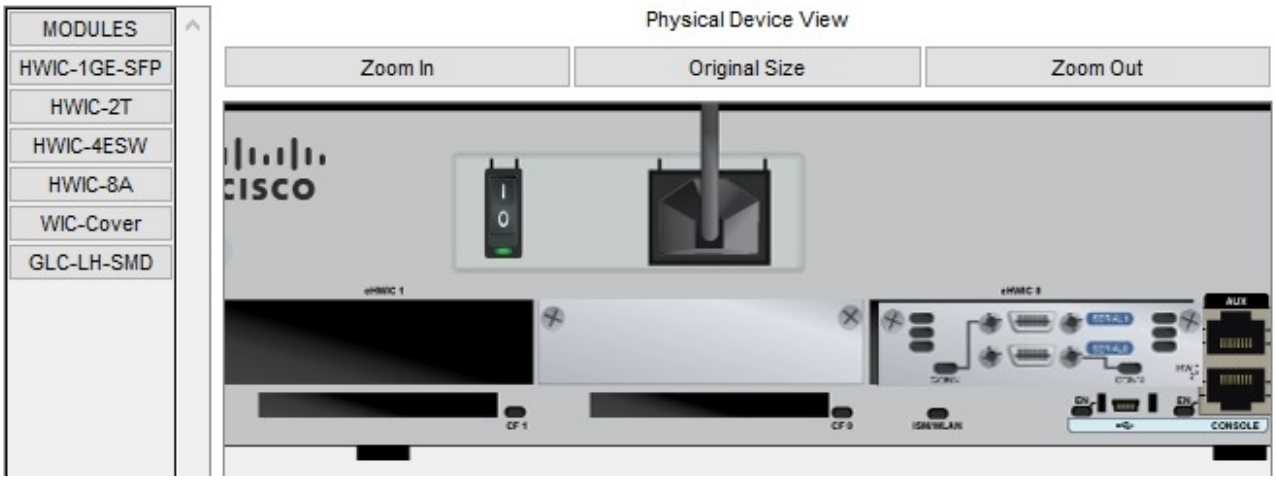
In this first section, we begin by configuring a network in Packet Tracer, establishing connections between devices, assigning IP addresses, and configuring routers to ensure network communication.

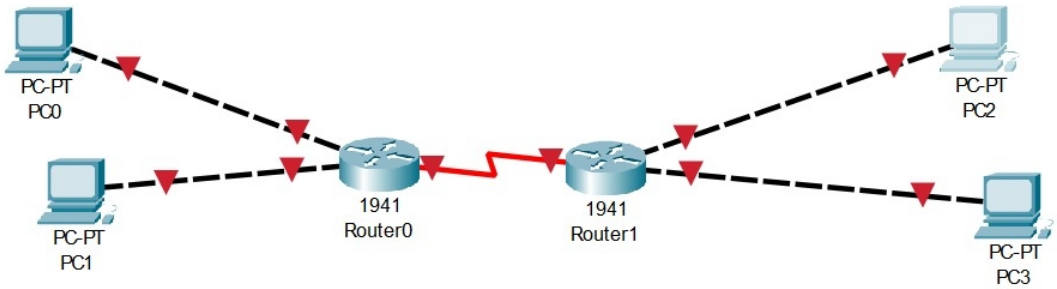
ROUTER SIMULATION

To begin, we utilize the same configuration and distribution of elements in the diagram in our application.

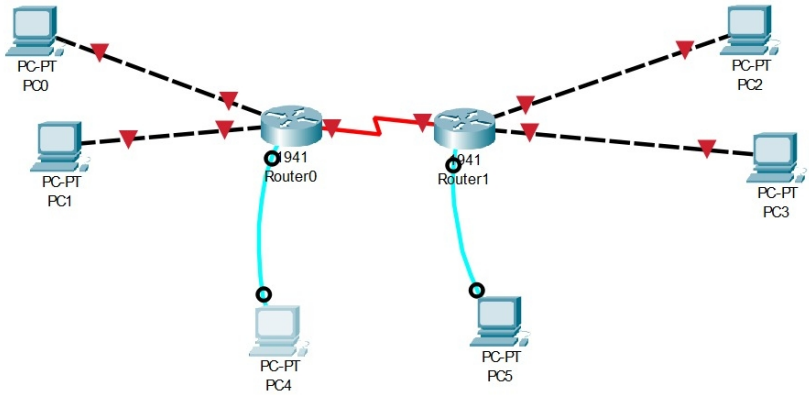


However, we still need to configure the cable between the routers. To do this, we use the physical view of the device, where we add the serial cable module, and then connect it. This shows that it's connected correctly.

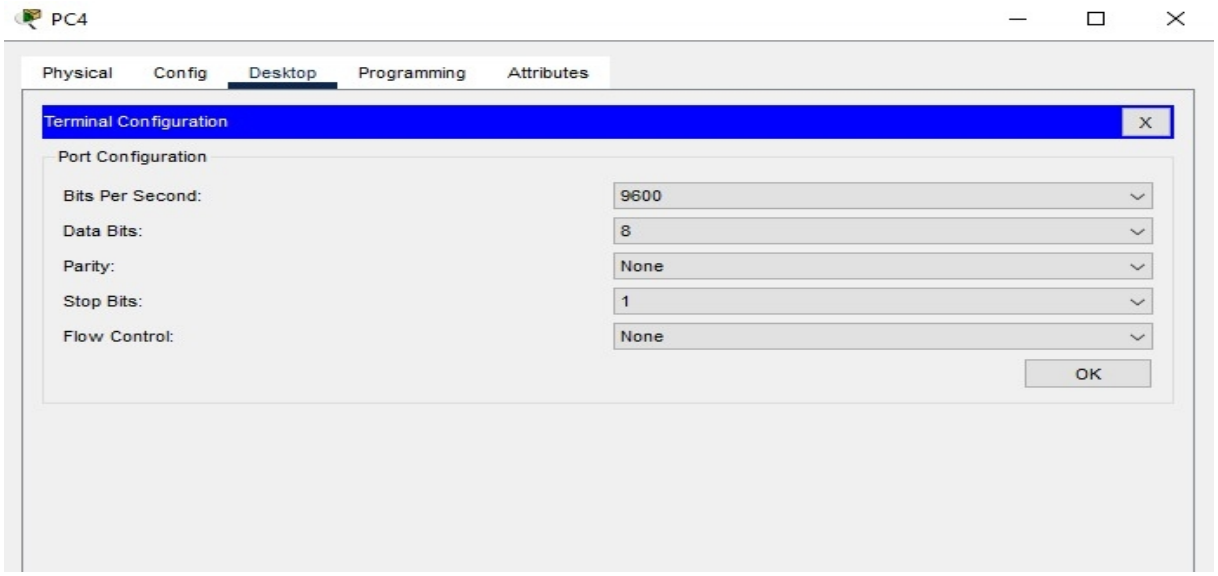




We are going to connect two computers to the routers with a console mode, the serial port is RS-232 for each device.



With the infrastructure in place, we open the device and go to the Desktop tab, leaving the settings unchanged. Then, open the router window and select the option to configure manually, get into the console window in privileged mode.




```
Image text-base: 0x2100F918, data-base: 0x24729040

This product contains cryptographic features and is subject to United
States and local country laws governing import, export, transfer and
use. Delivery of Cisco cryptographic products does not imply
third-party authority to import, export, distribute or use encryption.
Importers, exporters, distributors and users are responsible for
compliance with U.S. and local country laws. By using this product you
agree to comply with applicable laws and regulations. If you are unable
to comply with U.S. and local laws, return this product immediately.

A summary of U.S. laws governing Cisco cryptographic products may be found at:
http://www.cisco.com/wwl/export/crypto/tool/stqrg.html

If you require further assistance please contact us by sending email to
export@cisco.com.

Cisco CISCO1941/K9 (revision 1.0) with 491520K/32768K bytes of memory.
Processor board ID FTX152400KS
2 Gigabit Ethernet interfaces
2 Low-speed serial(sync/async) network interface(s)
DRAM configuration is 64 bits wide with parity disabled.
255K bytes of non-volatile configuration memory.
249856K bytes of ATA System CompactFlash 0 (Read/Write)


         --- System Configuration Dialog ---

Would you like to enter the initial configuration dialog? [yes/no]: n


Press RETURN to get started!



Router>enable
Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#
```

In this terminal view, we'll configure the specific settings for each router, based on the commands provided in the guide, it means the routing of each router. The final result is as follows:

- Router 0:

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#banner motd "For exclusive use by RECO students"
Router(config)#line console 0
Router(config-line)#logging synchronous
Router(config-line)#exit
Router(config)#no ip domain-lookup
Router(config)#interface GigabitEthernet0/0
Router(config-if)#description Conectado a la red de estudiantes
Router(config-if)#description Conectado a la tercera red de estudiantes
Router(config-if)#exit
Router(config)#interface GigabitEthernet0/1
Router(config-if)#description Conectado a la segunda red de estudiantes
Router(config-if)#exit
Router(config)#interface Serial0/0/0
Router(config-if)#description Conexion serial entre routers
Router(config-if)#exit
Router(config)#enable secret ClaveE
Router(config)#line console 0
Router(config-line)#password ClaveC
Router(config-line)#login
Router(config-line)#exit
Router(config)#line vty 0 4
Router(config-line)#password ClaveT
Router(config-line)#login
Router(config-line)#exit
Router(config)#exit
Router#
%SYS-5-CONFIG_I: Configured from console by console

Router#write memory
Building configuration...
[OK]
```

- Router 1:

```
Router(config-if)#hostname David
David(config)#banner motd "For exclusive use by RECO students"
David(config)#line console 0
David(config-line)#loggin syncronous
                                  ^
% Invalid input detected at '^' marker.

David(config-line)#loggin synchronous
David(config-line)#exit
David(config)#
David(config)#no ip domain-lookup
David(config)#interface Giga
David(config)#interface GigabitEthernet0/0
David(config-if)#description Conectado a la red de estudiantes
David(config-if)#exit
David(config)#interface
David(config)#interface
David(config)#interface Gi
David(config)#interface GigabitEthernet0/1
David(config-if)#description Conectado a la segunda red
David(config-if)#exit
David(config)#enable secre ClaveE
David(config)#enable secret ClaveE
David(config)#line console 0
David(config-line)#pasword ClaveC
                        ^
% Invalid input detected at '^' marker.

David(config-line)#password ClaveC
David(config-line)#login
David(config-line)#exit
David(config)#line vty 0 4
David(config-line)#password ClaveT
David(config-line)#login
David(config-line)#exit
David(config)#
```

Now, we validate the internal connections with a test. We found that they are communicating correctly at the internal connection level, that is, via the LAN.

```
Cisco Packet Tracer PC Command Line 1.0
C:\>tracert 72.0.0.2

Tracing route to 72.0.0.2 over a maximum of 30 hops:

  1   0 ms      0 ms      2 ms      132.18.0.1
  2   *         1 ms      0 ms      72.0.0.2

Trace complete.

C:\>ping 72.0.0.2

Pinging 72.0.0.2 with 32 bytes of data:

Reply from 72.0.0.2: bytes=32 time<1ms TTL=127
Reply from 72.0.0.2: bytes=32 time<1ms TTL=127

Ping statistics for 72.0.0.2:
    Packets: Sent = 2, Received = 2, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms

Control-C
^C
C:\>
```

In order for routers to be able to communicate with each other, connected devices are routed based on the interface that was given to us. This process, for each router where a kind of routing table of the network that we create is manually indicated

- Router 0:

```
Router(config-if)#ip route 73.0.0.0 255.0.0.0 20.0.0.2
Router(config)#ip route 73.0.0.0 255.0.0.0 20.0.0.2
Router(config)#ip route 74.0.0.0 255.0.0.0 20.0.0.2
Router(config)#ip route 75.0.0.0 255.0.0.0 20.0.0.2
Router(config)#ip route 154.126.0.0 255.255.0.0 20.0.0.2
Router(config)#ip route 154.127.0.0 255.255.0.0 20.0.0.2
Router(config)#ip route 154.128.0.0 255.255.0.0 20.0.0.2
Router(config)#ip route 154.129.0.0 255.255.0.0 20.0.0.2
Router(config)#
```

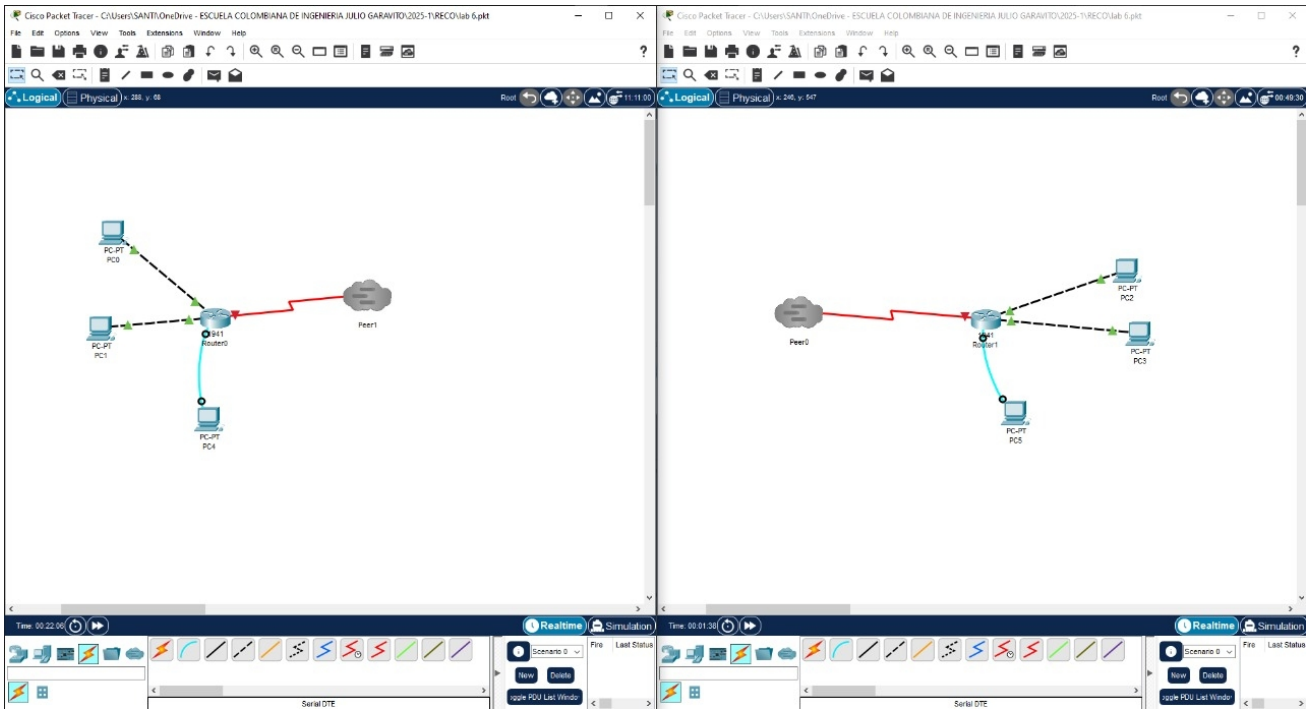
- Router 1:

```
Router(config-if)#ip route 132.18.0.0 255.255.0.0 20.0.0.1
Router(config)#ip route 132.19.0.0 255.255.0.0 20.0.0.1
Router(config)#ip route 132.20.0.0 255.255.0.0 20.0.0.1
Router(config)#ip route 132.21.0.0 255.255.0.0 20.0.0.1
Router(config)#ip route 72.0.0.0 255.0.0 20.0.0.1
                                 ^
% Invalid input detected at '^' marker.

Router(config)#ip route 72.0.0.0 255.0.0.0 20.0.0.1
Router(config)#
```

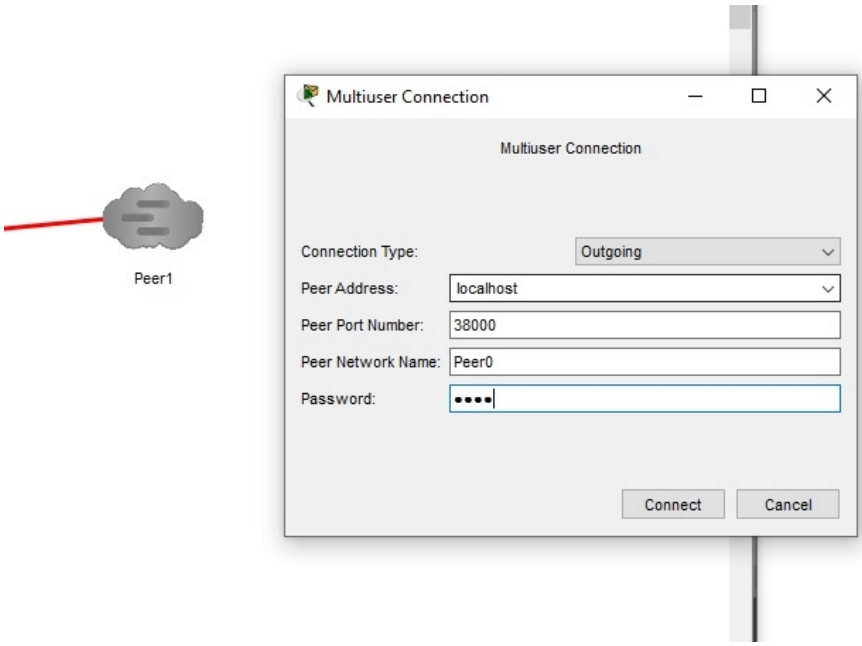
Finally, we need to validate that the devices are accessible and can connect to each other. Therefore, we use the same LAN connection test, but apply it to connecting two devices that differ in their router connections, thus confirming that our network is communicable.

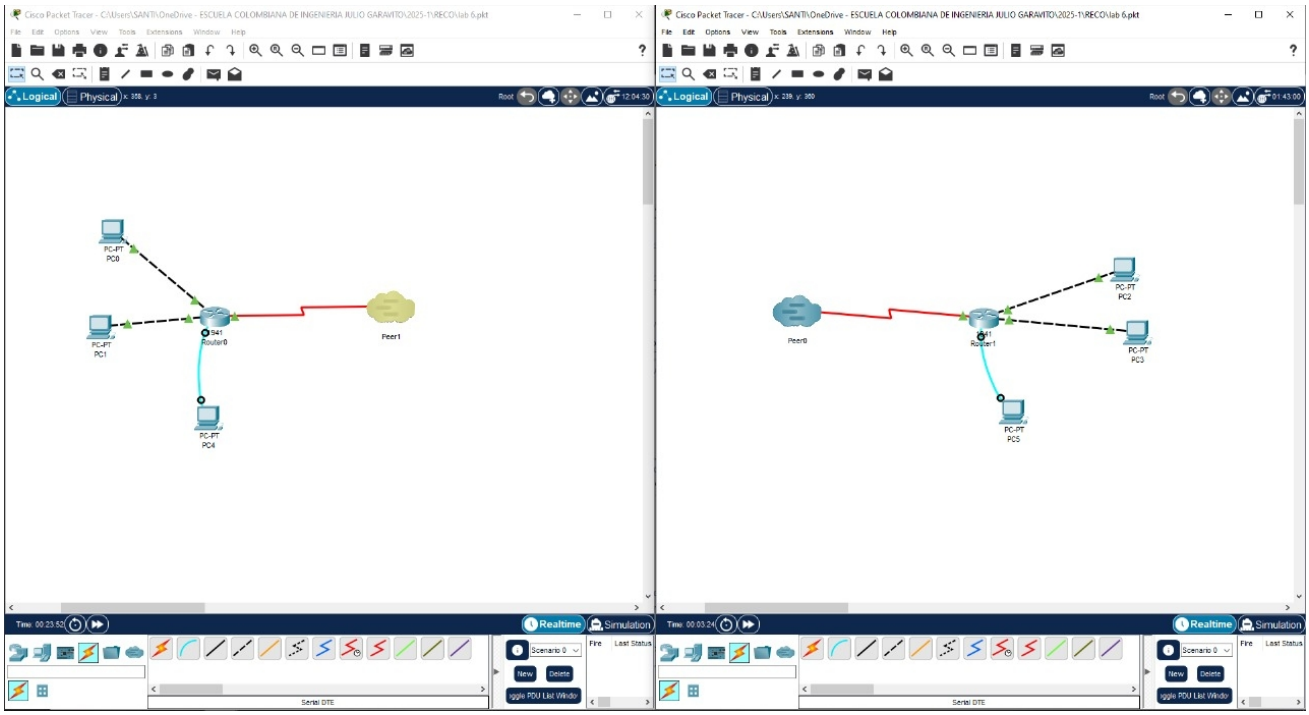
```
Tracing route to 154.126.0.2 over a maximum of 30 hops:

  1   0 ms      0 ms      0 ms      132.18.0.1
  2   1 ms      0 ms      1 ms      20.0.0.2
  3   *         3 ms      5 ms      154.126.0.2

Trace complete.

C:\>ping 154.126.0.2

Pinging 154.126.0.2 with 32 bytes of data:

Reply from 154.126.0.2: bytes=32 time=11ms TTL=126
Reply from 154.126.0.2: bytes=32 time=1ms TTL=126
Reply from 154.126.0.2: bytes=32 time=1ms TTL=126

Ping statistics for 154.126.0.2:
    Packets: Sent = 3, Received = 3, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 1ms, Maximum = 11ms, Average = 4ms

Control-C
^C
```

To finish this part, we create another file with the same settings as the original, as we want to use the multi-user option and connect to a simulation via the internet. Therefore, we place both setups side by side for better viewing.

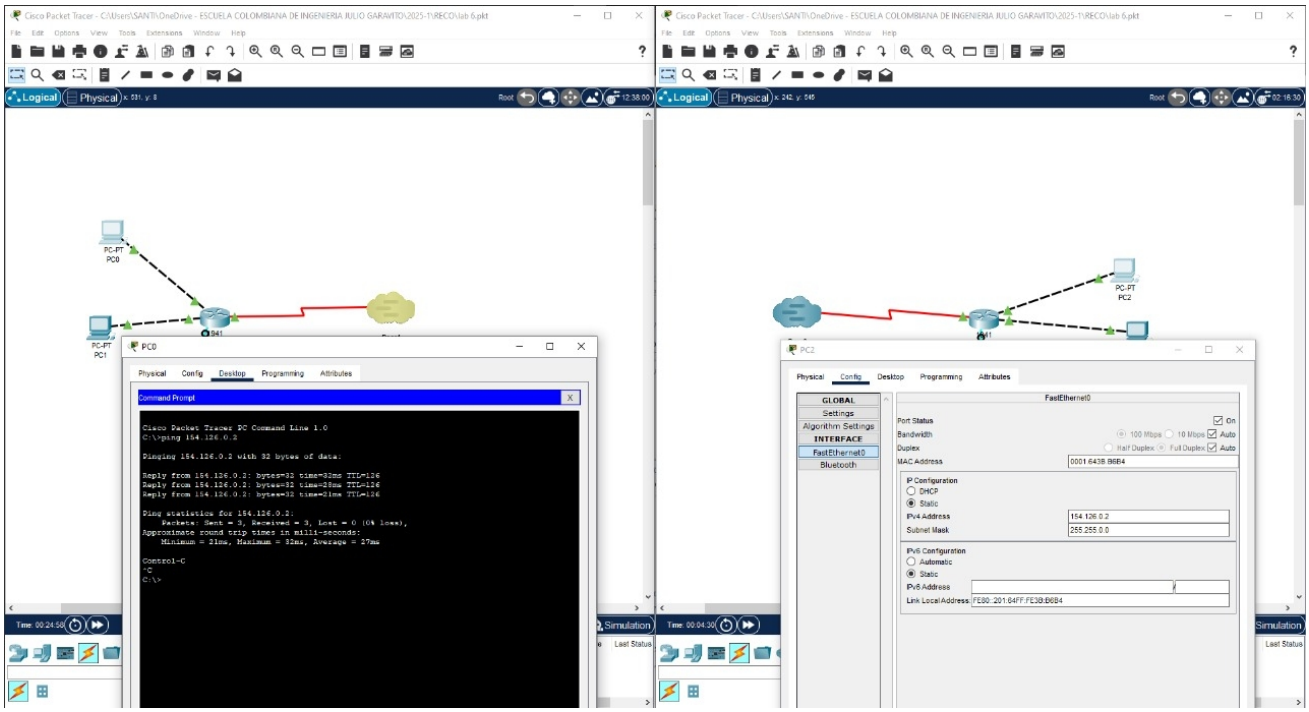


Now we are going to activate the multi-user connection by entering the credentials of one of the files, so that visually it is possible to notice that they are already linked through the cloud and their respective color.





Finally, we'll validate that the networks can communicate with each other. To do this, we ping from one computer in the first file to another in the second. This should work, assuming there are no other computers with the same IP address.



BASE SOFTWARE INSTALLATION

In this section, we install and configure web servers in various operating environments to provide web hosting services. Configurations are tested by creating test websites and ensuring key aspects such as automatic service startup and accessibility from other machines on the network. Additionally, we implement DNS services to facilitate access to servers via domain names, improving management and usability within the IT infrastructure.

WEB SERVICE INSTALATION

a) To install the Apache Web service on NetBSD, the default package installer was be used. Therefore, we decided to **not** download the package binary and install it using the `***make***` too, because it's unnecesary. To do this installation, we will use the `***pkgin install apache***` command to download the apache's package complete.

```
apache-2.4.62nb3: copying /usr/pkg/share/examples/httpd/mime.types to /usr/pkg/e
tc/httpd/mime.types
===========================================================================
The following files should be created for apache-2.4.62nb3:

        /etc/rc.d/apache (m=0755)
            [/usr/pkg/share/examples/rc.d/apache]

===========================================================================
===========================================================================
$NetBSD: MESSAGE,v 1.2 2014/02/22 17:28:34 ryoon Exp $

After apache-2.4.3, --enable-mpms-shared='event worker prefork' is
passed to configure script, then these multi-process model is built
and you can select the model in configuraton file.

The mod_cgi.so module conflicts with non-prefork multi-process model,
and mod_cgi.so module is not built anymore.
You can use mod_cgid.so module instead.
===========================================================================
pkg_install warnings: 1, errors: 0
pkg_install error log can be found in /var/db/pkgin/pkg_install-err.log
reading local summary...
processing local summary...
net#
```

With the service already installed, we will initialize it with the `***apachectl start***` command and validate its operation by obtaining the PID within the system with `***/etc/rc.d/apache status***`.

```
net# apachectl start
net# /etc/rc.d/apache
Usage: /etc/rc.d/apache [fast|force|one](start stop restart rcvar reload status
poll)
net# /etc/rc.d/apache status
apache is running as pid 686.
net#
```

b) To install Nginx on Slackware, it was necessary to install the binaries using the `***make***` tool, since the system's own package manager does not work correctly to install this package. To do this, we'll use the `***wget https://nginx.org/download/nginx-1.25.3.tar.gz***` command to download the necessary files.

```
root@darkstar:~# wget https://nginx.org/download/nginx-1.25.3.tar.gz
--2025-04-03 14:41:28--  https://nginx.org/download/nginx-1.25.3.tar.gz
Resolving nginx.org (nginx.org)... 52.58.199.22, 3.125.197.172, 64:ff9b::37d:c5ac, ...
Connecting to nginx.org (nginx.org)|52.58.199.22|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1216580 (1.2M) [application/octet-stream]
Saving to: 'nginx-1.25.3.tar.gz'

nginx-1.25.3.tar.gz     100%[===============================>]   1.16M  1.03MB/s    in 1.1s

2025-04-03 14:41:30 (1.03 MB/s) - 'nginx-1.25.3.tar.gz' saved [1216580/1216580]

root@darkstar:~# ls
123      Documents  Music     Public     Videos               postgresql-15.12
Desktop  Downloads  Pictures  Templates  nginx-1.25.3.tar.gz
root@darkstar:~# tar -x
```

Next, we'll move the files to a folder within the `***/etc/local*** ` directory after unpackage it whit the tar tool `***tar -xvf nginx-1.25.3.tar.gz***`, as that's where we store the files.

```
nginx-1.25.3/auto/cc/msvc
nginx-1.25.3/auto/cc/name
nginx-1.25.3/auto/cc/owc
nginx-1.25.3/auto/cc/sunc
root@darkstar:~# cd nginx-1.25.3
root@darkstar:~/nginx-1.25.3# ls
CHANGES  CHANGES.ru  LICENSE  README  auto  conf  configure  contrib  html  man  src
root@darkstar:~/nginx-1.25.3#
```

```
root@darkstar:~/nginx-1.25.3# mkdir -p /usr/local/nginx
root@darkstar:~/nginx-1.25.3# cd -r * /usr/local/nginx
bash: cd: -r: invalid option
cd: usage: cd [-L|[-P [-e]] [-@]] [dir]
root@darkstar:~/nginx-1.25.3# cp -r * /usr/local/nginx
root@darkstar:~/nginx-1.25.3#
```

Then we will configure the address where we will mainly install the application, using the command `***./configure/ --prefix=usr/local/nginx –with-https_ssl_module –with-http_gzip_static_module --with-pcre***`, to finally execute the `***make***` command to prepare the installation package, and then we do it with the `***make install***` command.

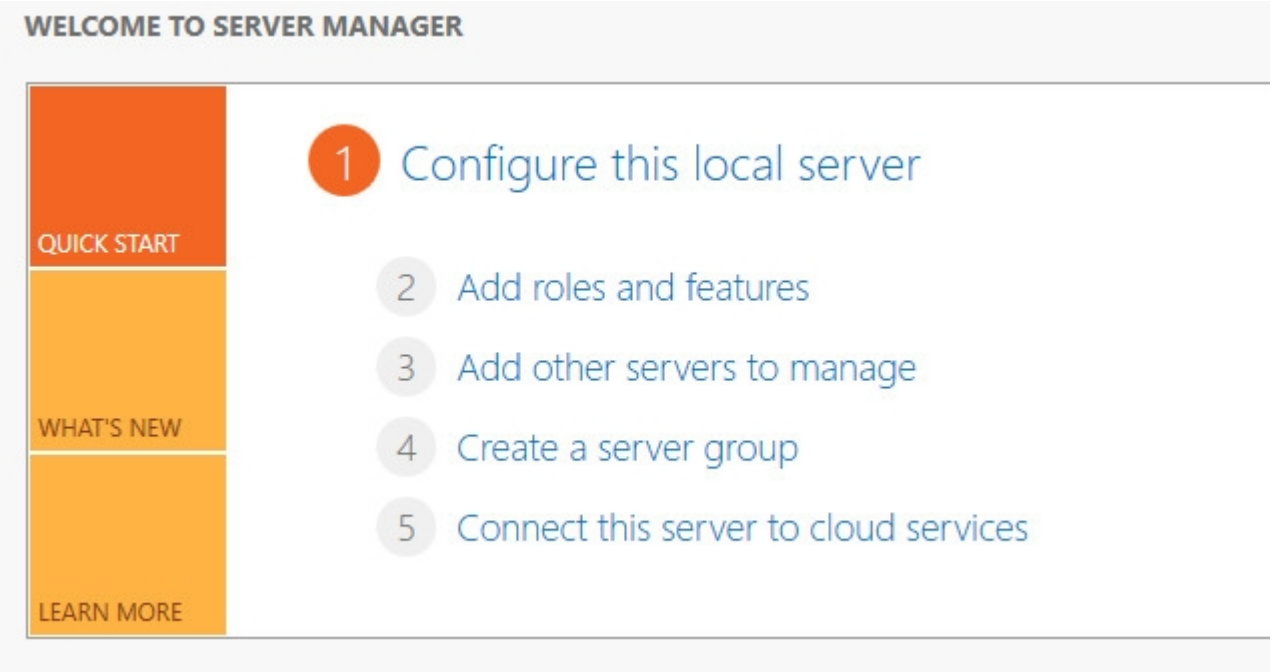
```
root@darkstar:~/nginx-1.25.3# ./configure --prefix=/usr/local/nginx --with-http_ssl_module --wit
h-http_gzip_static_module --with-pcre
checking for OS
 + Linux 5.15.19-smp i686
checking for C compiler ... found
 + using GNU C compiler
 + gcc version: 11.2.0 (GCC)
checking for gcc -pipe switch ... found
checking for -Wl,-E switch ... found
```

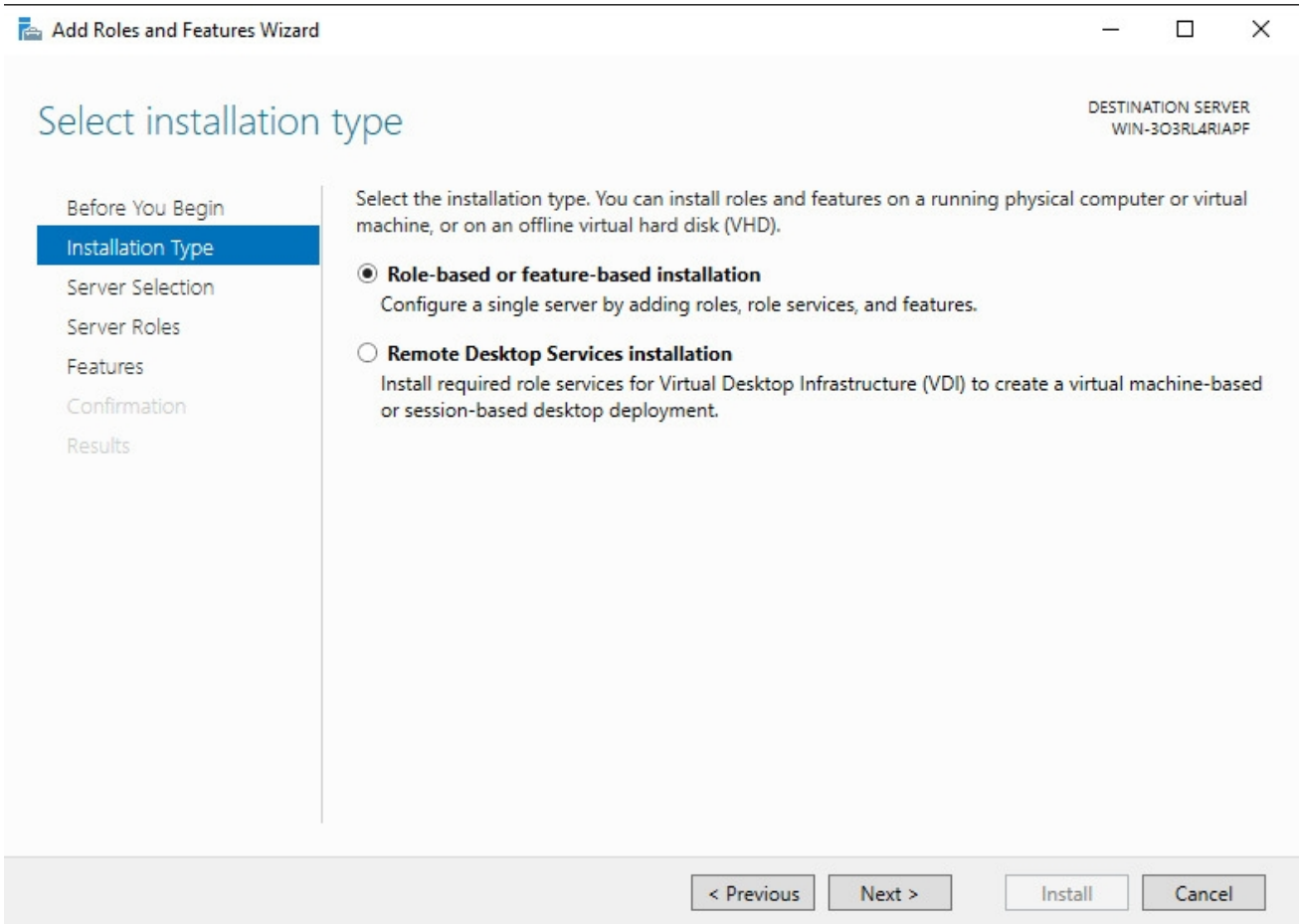
```
root@darkstar:~/nginx-1.25.3# make install
make -f objs/Makefile install
make[1]: Entering directory '/root/nginx-1.25.3'
cc -c -pipe  -O -W -Wall -Wpointer-arith -Wno-unused-parameter -Werror -g  -I src/core -I src/ev
ent -I src/event/modules -I src/event/quic -I src/os/unix -I objs \
        -o objs/src/core/nginx.o \
        src/core/nginx.c
cc -c -pipe  -O -W -Wall -Wpointer-arith -Wno-unused-parameter -Werror -g  -I src/core -I src/ev
ent -I src/event/modules -I src/event/quic -I src/os/unix -I objs \
        -o objs/src/core/ngx_log.o \
        src/core/ngx_log.c
cc -c -pipe  -O -W -Wall -Wpointer-arith -Wno-unused-parameter -Werror -g  -I src/core -I src/ev
ent -I src/event/modules -I src/event/quic -I src/os/unix -I objs \
        -o objs/src/core/ngx_palloc.o \
        src/core/ngx_palloc.c
cc -c -pipe  -O -W -Wall -Wpointer-arith -Wno-unused-parameter -Werror -g  -I src/core -I src/ev
ent -I src/event/modules -I src/event/quic -I src/os/unix -I objs \
        -o objs/src/core/ngx_array.o \
        src/core/ngx_array.c
```

Then, we start the sevice in order to enable it and use it with the command `***/user/local/nginx/sbin/nginx***`.

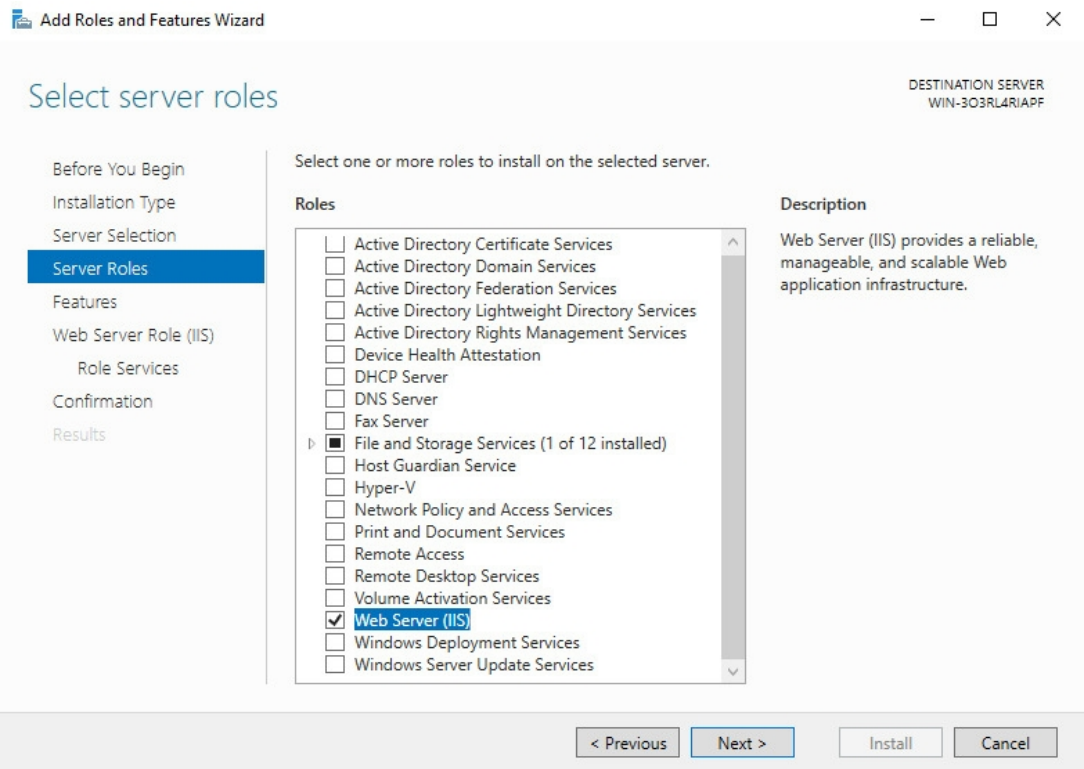
```
root@darkstar:~/nginx-1.25.3# /usr/local/nginx/sbin/nginx -v
nginx version: nginx/1.25.3
root@darkstar:~/nginx-1.25.3# /usr/local/nginx/sbin/nginx start
nginx: invalid option: "start"
root@darkstar:~/nginx-1.25.3# /usr/local/nginx/sbin/nginx
```

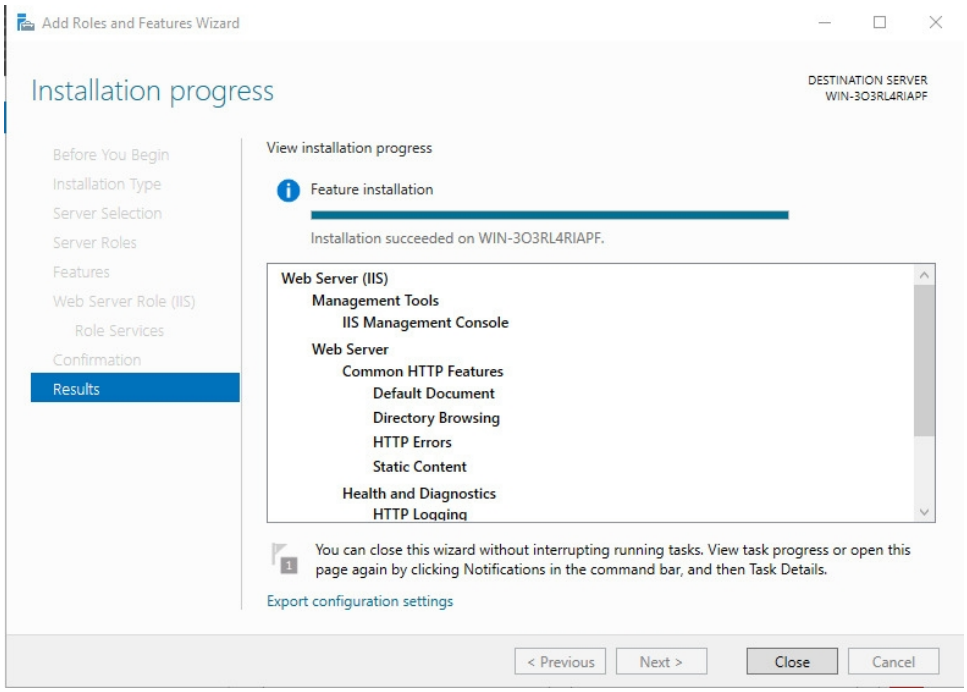
c) In the case of Windows Server, there's a default service on the system, a web service named ISS. Initially, we'll go to the Windows Server application and click on the section labeled "Add roles and features" and "Role-based."


WELCOME TO SERVER MANAGER
QUICK START
WHAT'S NEW
LEARN MORE
1 Configure this local server
2 Add roles and features
3 Add other servers to manage
4 Create a server group
5 Connect this server to cloud services


Add Roles and Features Wizard
Select installation type
DESTINATION SERVER
WIN-3O3RL4RIAPF
Before You Begin
Installation Type
Server Selection
Server Roles
Features
Confirmation
Results
Select the installation type. You can install roles and features on a running physical computer or virtual machine, or on an offline virtual hard disk (VHD).
Role-based or feature-based installation
Configure a single server by adding roles, role services, and features.
Remote Desktop Services installation
Install required role services for Virtual Desktop Infrastructure (VDI) to create a virtual machine-based or session-based desktop deployment.
< Previous
Next >
Install
Cancel

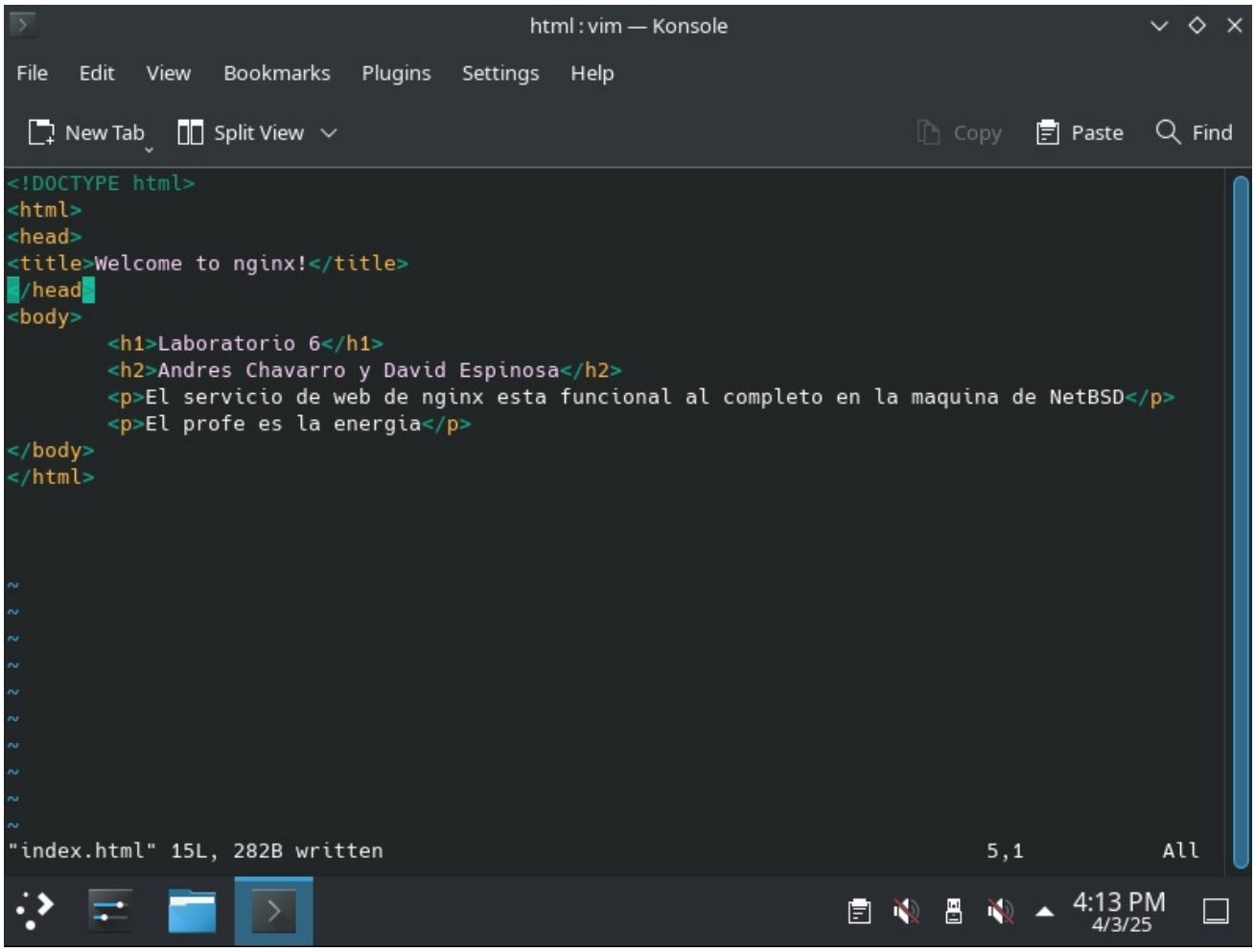
Next, we'll look for the box that indicates the ISS service, select it, and leave everything else as default, including subsequent screens, until the download starts and finishes the instalation of the process.





d) For all the machines we configure, we used the same super-basic page style that only serves to show that the service ran correctly.

- Slackware: We go to the `***/usr/local/nginx/html/***` directory where the page shown in the example is located when running the service, then we replace it with the following code.

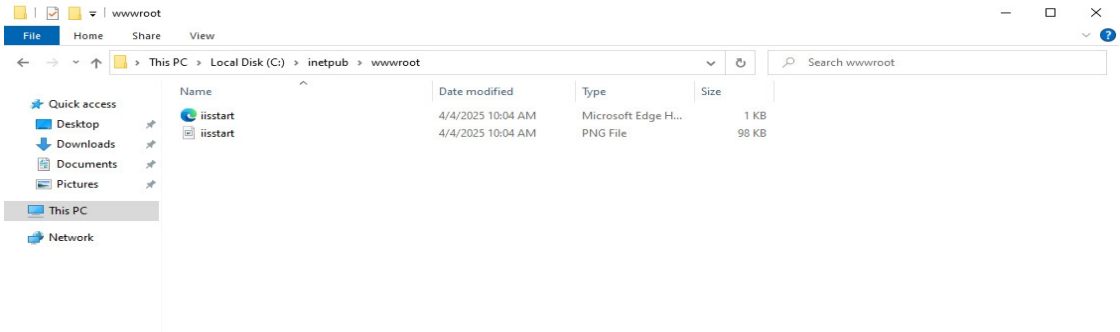


- NetBDS: We go to the `***/usr/pkg/share/httpd/htsdocs/***` directory where the page shown in the example is located when running the service, then we replace it with the following code.

```
<!DOCTYPE html>
<html lang="es">
<head>
        <title>laboratorio 6</title>
</head>

<body>
        <h1>Laboratorio 6</h1>
        <h2>Andres Chavarro y David Espinosa</h2>
        <p>El servicio web de apache o nginx esta funcionando correctamente</p>
        <p>El profe es la energia</p>
</body>
</html>
~
~
~
~
~
~
~
~
~
~
~
"index.html" 13L, 273B written                                  11,1-8        All
```

- Windows: In this case, we create a file containing our demo web page. Then, we place it in the `***C: \ inetput \ wwwroot*** ` folder with the name index, so the service recognizes it as part of its system.

```
*iisstart - Notepad
File  Edit  Format  View  Help
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Laboratorio 6</title>
</head>
<body>
    <h1>Laboratorio 6</h1>
    <h2>Andrés Chavarro y David Espinosa</h2>
    <p>Windows sigue siendo peor para solo inicar cualquier servicio jsajsajsas.</p>
    <p>El profe es la energia</p>
</body>
</html>
```

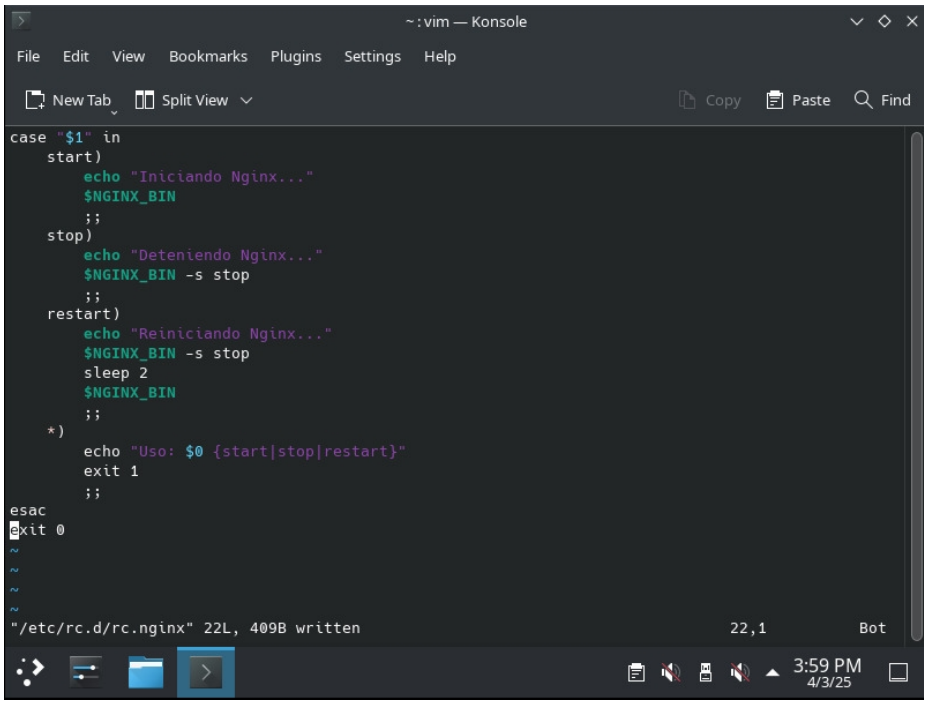


e) The way to start these services varies depending on the operating system being used. First, we configure NetBSD, which only requires creating a boot file indicating that the service will run from the moment the device is powered on.

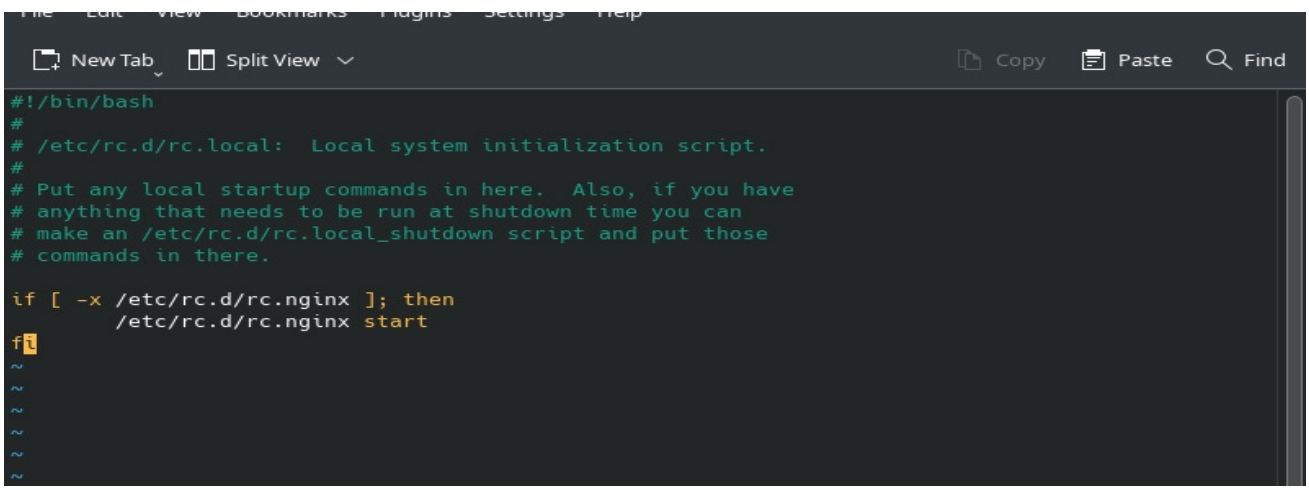
```
net# cd /etc/rc.conf.d/
net# vim apache
```

```
apache=YES
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
"apache" 1L, 11B                                    1,10          All
```

In the case of Slackware, we have to create the script that allows the service to run. To do this, we create a file that uses the command we already saw to start it. Then, move it to the directory `***/etc/rc.d/rc.nginx***`.



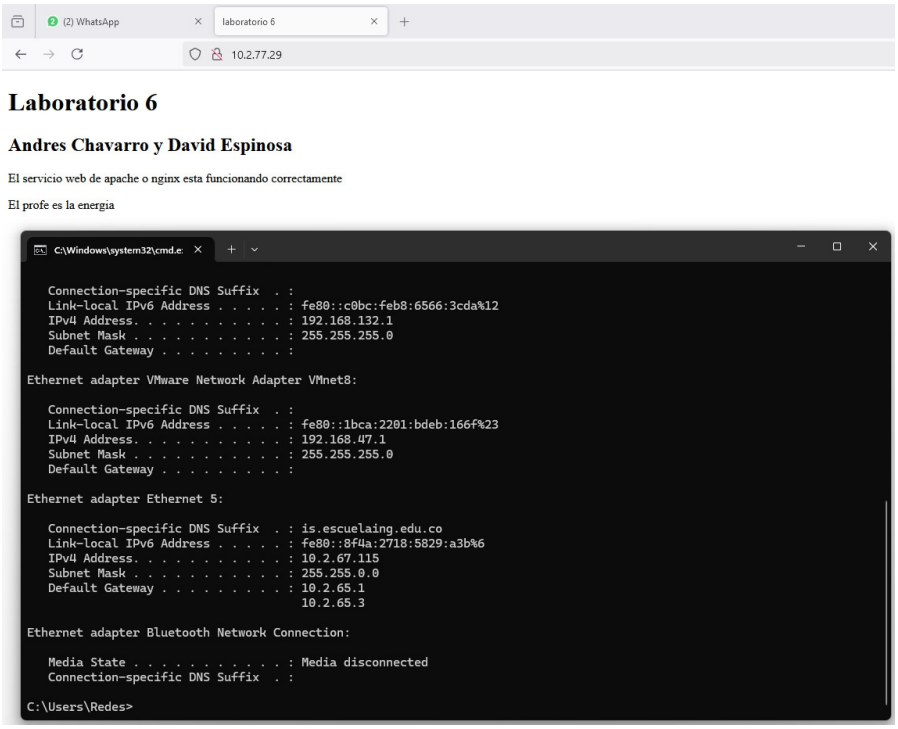
Finally, we place a few lines of code in the `***/etc/rc.d/rc.local*** ` file in the startup file, which are only executed when the system is turned on.



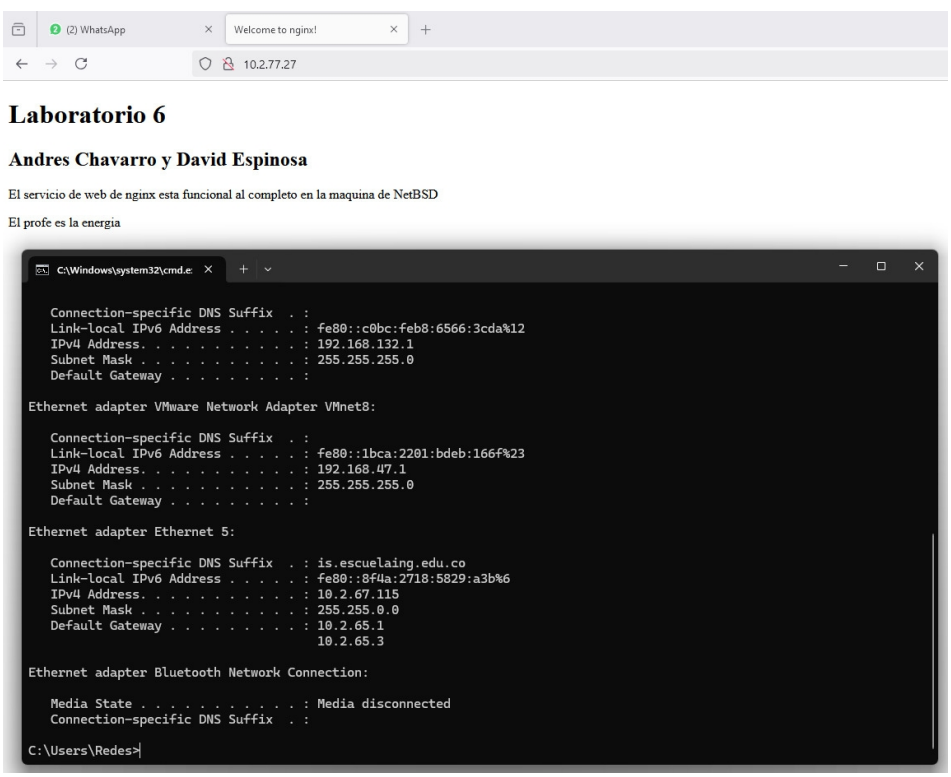
In case of Windows Server, the service starts when the operating system boots, so no further configuration is required.

f) To validate that each service is running correctly, we use the computer outside the machine and try to enter through the IP of each of the machines, as seen in the following images.

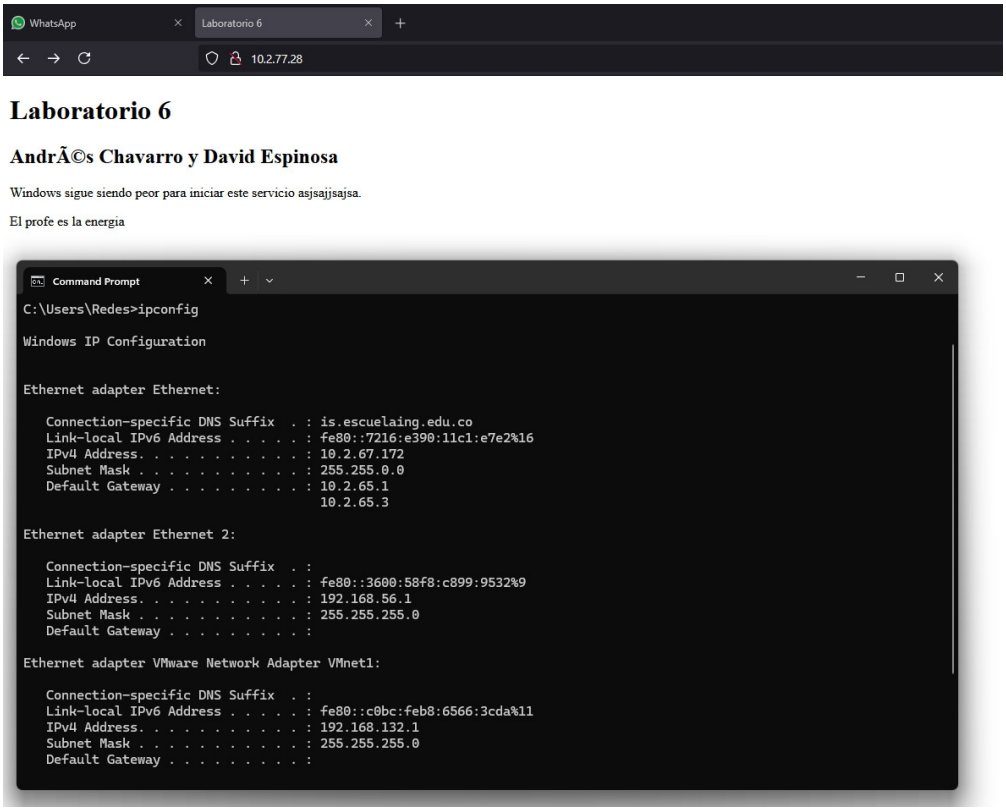
- Slackware:



- NerBSD:



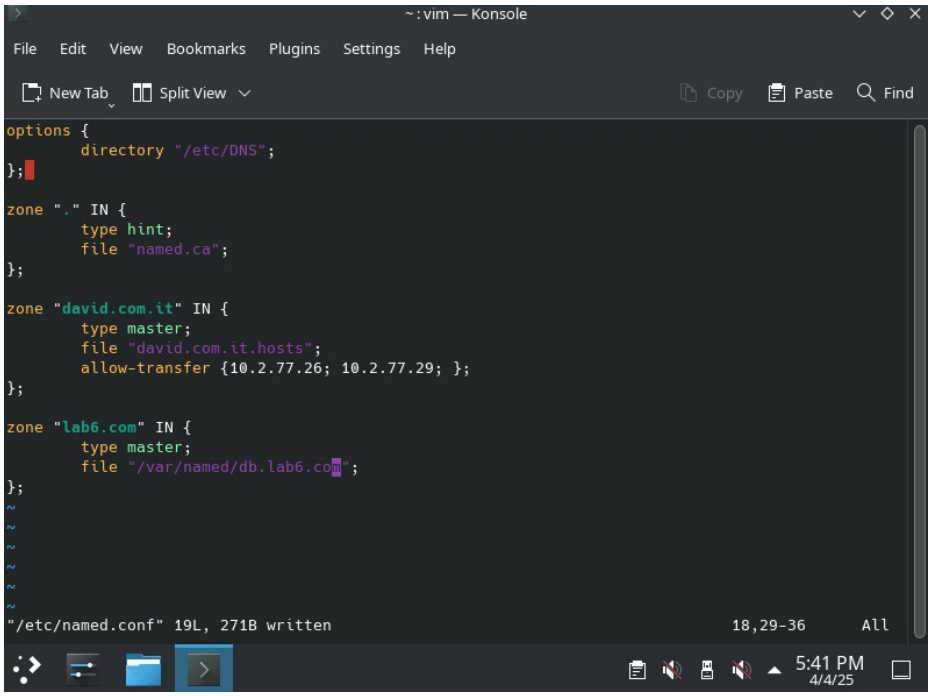
- Windows Server:



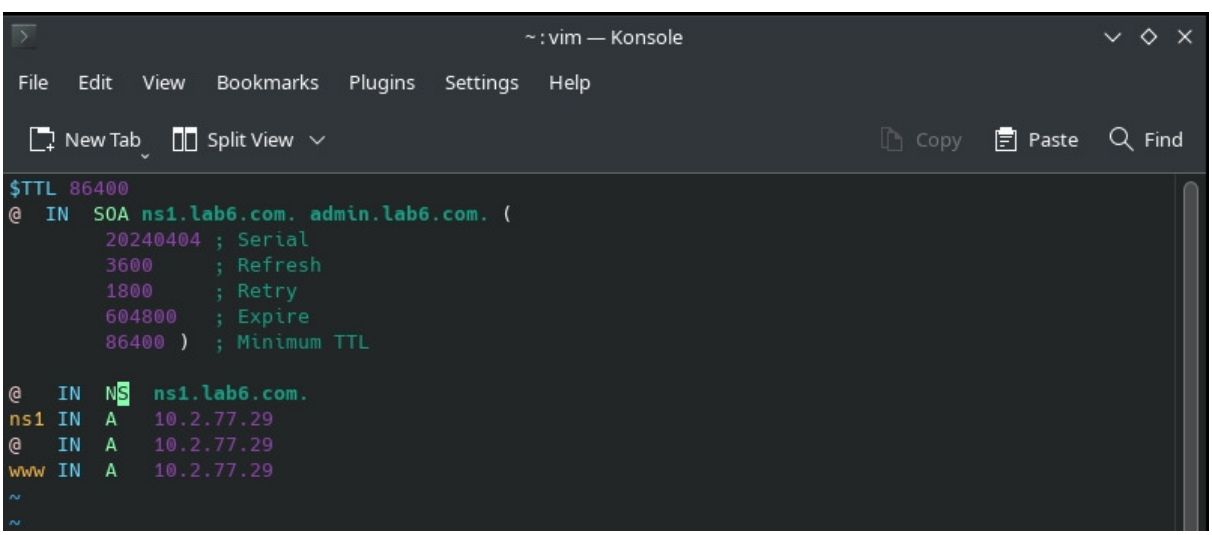
g) To assign a custom IP address using the DNS service, we need to configure each system separately, as shown below.

- Slackware and NetBSD:

  The same bind package configuration is used for both systems. For practical reasons, only the Slackware configuration will be shown, as we'll use NetBSD for other configurations later. First, let's access the named configuration with the following command `***vim /etc/named.conf*** `.

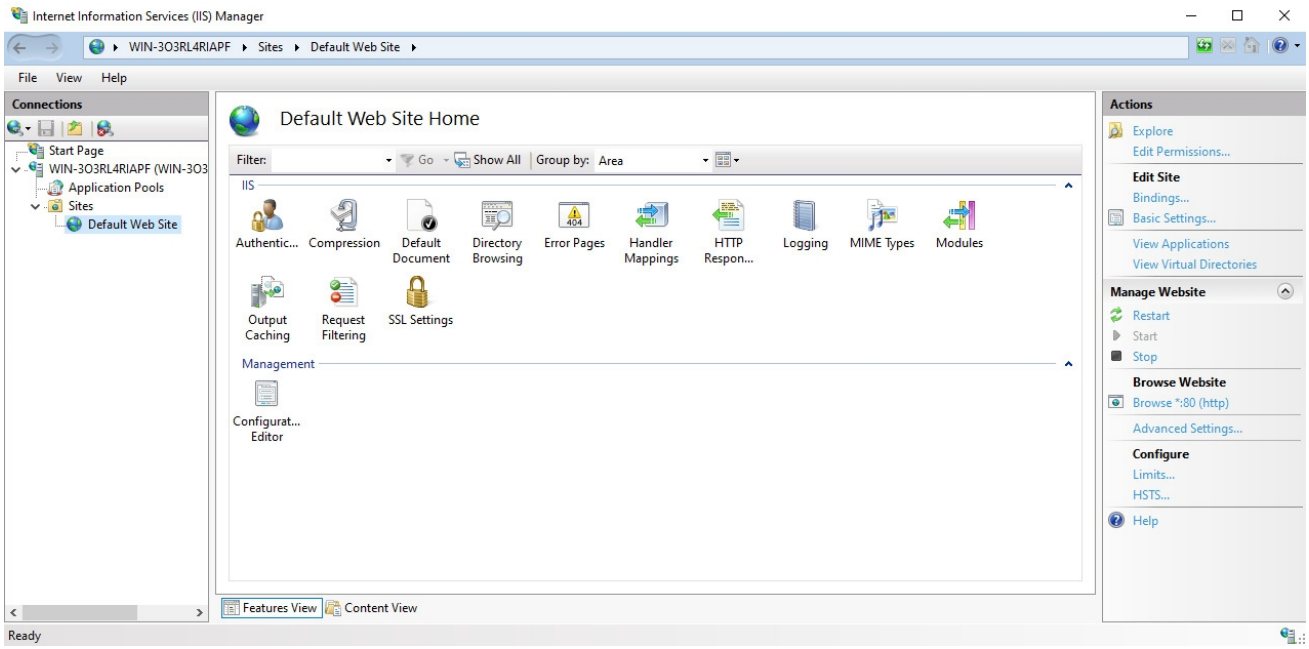


Enter the address you want your machine to have, followed by the master type for the page host and the configuration address. Next, we'll navigate to that configuration with the command `***vim /var/named/db.lab6.com***` and enter the IP address for the machine, in addition to the configurations seen in previous labs.

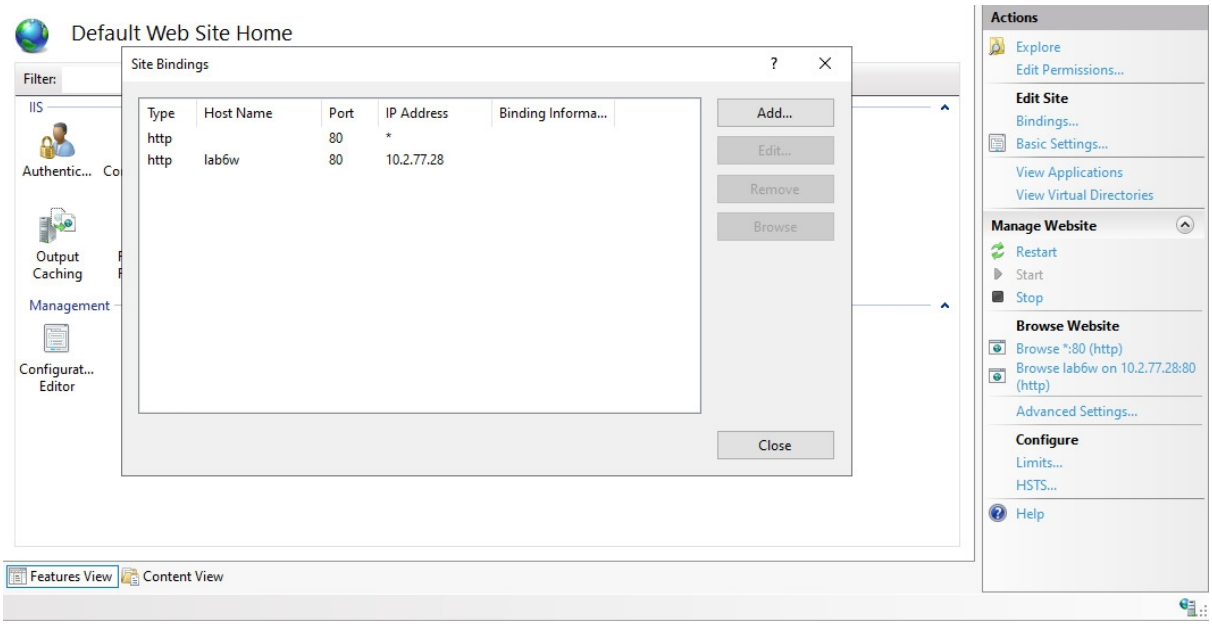


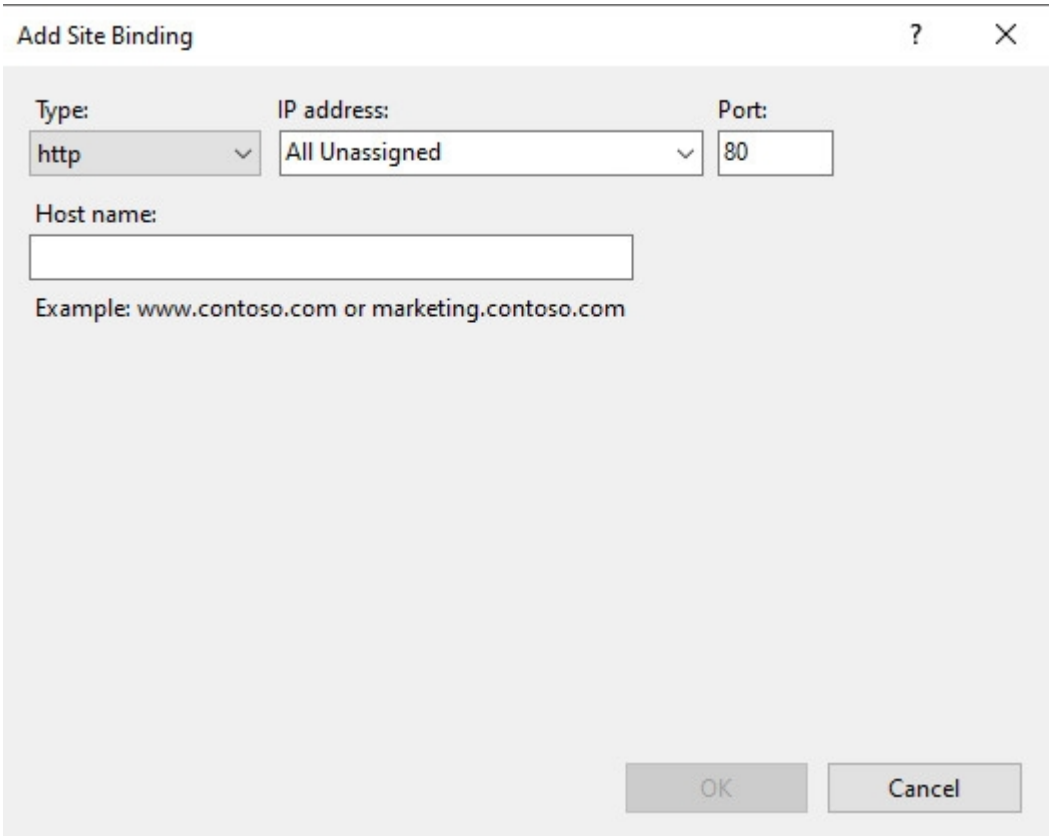
Now, from any device, we can access the declared IP as long as the DNS is set to the IP of the machine that is active and hosting the DNS.

- Windows: In the case of Windows, we have to open the application specified for the IIS service where we can set special configurations to access our page using a custom URL.



On the right side, we open a section named Bindings, where a menu drops down and allows us to add multiple URLs that can point to our app's IP or others we have in the service itself.





Once we have entered the address we want, we have to enable the connection from other devices by assigning a custom DNS service and connecting it at the lab computer.

HOSTING SERVICE CONFIGURATION

In this section, we will create multiple web pages that will be used by Apache Web Services. Each page will be assigned a unique URL. Therefore, it will be necessary to configure the DNS and Apache virtual hosts.

First, we go to the file `***/usr/pkg/etc/httpd/httpd.conf*** ` and look for the line that enables the virtual hosts, then we uncomment the line and we can save and continue.

```
#Include etc/httpd/httpd-mpm.conf

# Multi-language error messages
#Include etc/httpd/httpd-multilang-errordoc.conf

# Fancy directory listings
#Include etc/httpd/httpd-autoindex.conf

# Language settings
#Include etc/httpd/httpd-languages.conf

# User home directories
#Include etc/httpd/httpd-userdir.conf

# Real-time info on requests and configuration
#Include etc/httpd/httpd-info.conf

# Virtual hosts
Include etc/httpd/httpd-vhosts.conf

# Local access to the Apache HTTP Server Manual
#Include etc/httpd/httpd-manual.conf

# Distributed authoring and versioning (WebDAV)
<usr/pkg/etc/httpd/httpd.conf" 525L, 19737B written          497,1         95%
```

Second, we'll configure the hosts for each of the three addresses requested in the guide, indicating the addresses of the directories where the pages are created. To do this, go to the `***/usr/pkg/etc/httpd/httpd-vhost.conf*** ` document, save it, and continue.

```
<VirtualHost *:80>
    ServerAdmin webmaster@dummy-host2.example.com
    DocumentRoot "/usr/pkg/docs/dummy-host2.example.com"
    ServerName dummy-host2.example.com
    ErrorLog "/var/log/httpd/dummy-host2.example.com-error_log"
    CustomLog "/var/log/httpd/dummy-host2.example.com-access_log" common
</VirtualHost>


<VirtualHost *:80>
        ServerName network.andres.com.co
        DocumentRoot "/usr/pkg/share/httpd/network"
</VirtualHost>

<VirtualHost *:80>
        ServerName security.david.org.jp
        DocumentRoot "/usr/pkg/share/httpd/security"
</VirtualHost>

<VirtualHost *:80>
        ServerName systems.chavarro.com.cl
        DocumentRoot "/usr/pkg/share/httpd/systems"
</VirtualHost>
<kg/etc/httpd/httpd-vhosts.conf" 54L, 1830B written
net#
```

With the Apache configurations complete, we create the web pages in a very basic way after creating the directories indicated in the virtual host configurations. Then, we use the following commands to complete this process quickly and efficiently.

```
net# mkdir -p /usr/pkg/share/httpd/network
net# mkdir -p /usr/pkg/share/httpd/security
net# mkdir -p /usr/pkg/share/httpd/systems
net# cd /usr/pkg/share/httpd/
net# ls
build     error     htdocs    icons     manual    network   security  systems
net#
```

```
net# echo "<h1>Network Site by Andres</h1>" > /usr/pkg/share/httpd/network/index
.html
net# echo "<h1>Network Site by David</h1>" > /usr/pkg/share/httpd/security/index
.html
net# echo "<h1>Network Site by Chavarro</h1>" > /usr/pkg/share/httpd/systems/ind
ex.html
net#
```

At this point, we can restart the Apache service with the command `***service apache restart*** ` and proceed with configuring the domains via DNS. To do this, we go to the `***/etc/named.conf*** ` file and add the zones we'll use to fulfill the URLs requested by the guide.

```
zone "com.co" {
        type master;
        file "/etc/namedb/com.co.zone";
};

zone "org.jp" {
        type master;
        file "/etc/namedb/org.jp.zone";
};

zone "com.cl" {
        type master;
        file "/etc/namedb/com.cl.zone";
};

# example secondary server config:
#
# zone "Berkeley.EDU" {
#       type slave;
#       file "berkeley.edu.cache";
"/etc/named.conf" 60L, 881B written                    21,13-20      27%
```

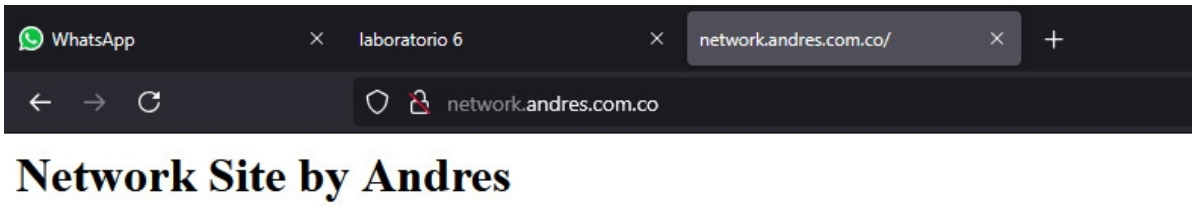
By going to the folder where the configurations for the previous zones were declared, we will create the configuration for each of these. The only thing that changes would be the domain to which access is assigned.

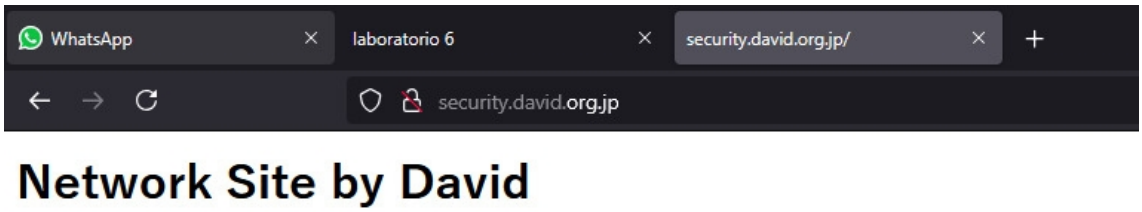
```
$TTL 86400
@    IN   SOA   ns1.andres.com.co. root.andres.com.co. (
          20240404   ; Serial
          3600       ; Refresh
          1800       ; Retry
          1209600    ; Expire
          86400 )    ; Minimum TTL

     IN   NS   ns1.andres.com.co.
ns1  IN   A    10.2.77.27
andres IN A 10.2.77.27
~
~
~
~
~
~
~
~
~
~
~
~
"/etc/namedb/com.co.zone" [New] 11L, 283B written              1,1           All
```

```
$TTL 86400
@   IN   SOA   ns1.chavarro.com.cl. root.chavarro.com.cl. (
          20240404   ; Serial
          3600       ; Refresh
          1800       ; Retry
          1209600    ; Expire
          86400 )    ; Minimum TTL

     IN   NS   ns1.chavarro.com.cl.
ns1  IN   A    10.2.77.27
chavarro IN A 10.2.77.27
~
~
~
~
~
~
~
~
~
~
~
~
"/etc/namedb/com.cl.zone" [New] 11L, 290B written              1,10          All
```

```
@   IN   SOA   ns1.david.org.jp. root.david.org.jp. (
          20240404    ; Serial
          3600        ; Refresh
          1800        ; Retry
          1209600     ; Expire
          86400 )     ; Minimum TTL

     IN   NS   ns1.david.org.jp.
ns1  IN   A    10.2.77.27
david IN A 10.2.77.27
~
~
~
~
~
~
~
~
~
~
~
~
~
"/etc/namedb/org.jp.zone" [New] 11L, 278B written
net#
```

Finally, we can restart the DNS service with the `***/etc/named onerestart*** ` command to load the configuration settings. Next, we'll go to the main computer and assign the DNS to the NetBSD machine's IP address, so we can launch our web pages with the assigned domains.





Network Site by Chavarro

CONCLUTIONS

Upon completing this lab, we achieved comprehensive network infrastructure configuration in Packet Tracer. Our efforts ensured seamless connectivity across devices through meticulous IP addressing, subnetting, and the implementation of routing strategies. This simulation provided invaluable insights into router functionality, including interface configuration, routing table management, and the deployment of static routes to effectively link diverse network segments.

In addition to the network configuration, we successfully installed and configured web servers across different operating systems, including both Windows and Linux environments. These servers were tested using custom web pages, which were made accessible from other hosts within the network, verifying correct configuration and connectivity. This step allowed us to understand the process of setting up essential services in a multi-platform environment and how to troubleshoot issues related to service availability.

Moreover, we configured the DNS service to resolve domain names to IP addresses, allowing users to access web servers using easy-to-remember names rather than numeric addresses. This significantly improved the usability and user experience within the network, demonstrating the importance of name resolution in real-world networks.

Overall, this lab reinforced a wide range of fundamental concepts in networking and server administration. It provided hands-on experience in designing and deploying a functional IT infrastructure, covering both client-server interactions and core network services. The practical knowledge gained through this lab is directly applicable to real-world scenarios and serves as a strong foundation for more advanced topics in networking and systems management

BIBLIOGRAFY

Anicas, M. (2014, julio 22). *How to install Nginx on CentOS 7*. Digitalocean.com; DigitalOcean. https://www.digitalocean.com/community/tutorials/how-to-install-nginx-on-centos-7

Configuración del routing entre VLAN con el uso de un router externo. (2024, marzo 5). Cisco. https://www.cisco.com/c/es_mx/support/docs/lan-switching/inter-vlan-routing/14976-50.html

Configuring IIS for web hosting on Windows: A step-by-step guide. (2025, febrero 23). Kamatera. https://www.kamatera.com/knowledgebase/configuring-iis-for-web-hosting-on-windows/

Documentation Group. (s/f). *Documentation: Apache HTTP server - the Apache HTTP server project*. Apache.org. Recuperado el 6 de abril de 2025, de https://httpd.apache.org/docs/

nginx documentation. (s/f). Nginx.org. Recuperado el 6 de abril de 2025, de https://nginx.org/en/docs/

(S/f). Cloudflare.com. Recuperado el 6 de abril de 2025, de https://www.cloudflare.com/learning/dns/what-is-dns/